REST API Mini Project

Employee Management System (EMS)

Configuring a Fake REST API using JSON Server and Testing Endpoints with Postman

Prepared by: Srakshin Chityala

Date: 11 July 2026

1. Introduction

ABC Technologies Pvt. Ltd. has more than 500 employees working across different departments such as IT, HR, Finance, Sales, and Administration. Currently, employee information is maintained in spreadsheets, which makes it difficult to search employee records, update employee details, delete employee details, and generate reports.

To solve these issues, the company plans to develop an Employee Management System (EMS) using Java Full Stack technologies (Java, Spring Boot, REST API, MySQL, HTML/CSS/JavaScript, React or Angular).

2. Objectives

Develop a REST API that enables administrators to:

- Add new employees
- View all employees
- Search employees by ID
- Update employee information
- Delete employees

3. Functional Requirements

Employee Module

Each employee record contains the following fields:

Field	Type
Employee ID	Integer
First Name	String
Last Name	String
Email	String
Mobile Number	String

4. REST API Design

The following REST endpoints were designed and implemented for the Employee resource:

Method	URL	Description
GET	/employees	Get all employees
GET	/employees/{id}	Get employee by ID
POST	/employees	Add employee
PUT	/employees/{id}	Update employee

PATCH	/employees/{id}	Partial update
DELETE	/employees/{id}	Delete employee

5. Environment Setup

The following tools were used to configure and test the Fake REST API:

- Node.js and npm — JavaScript runtime for running JSON Server
- JSON Server — used to simulate a REST API from a db.json file
- Postman — used to test all REST API endpoints

The db.json file was created with the employee schema (id, firstName, lastName, email, mobileNumber) and populated with sample employee records. The JSON Server was then started on port 3000 using the following command:

```
json-server --watch db.json --port 3000
```

5.1 JSON Server Running on Port 3000

The terminal below confirms that JSON Server started successfully on port 3000, exposing the /employees endpoint.

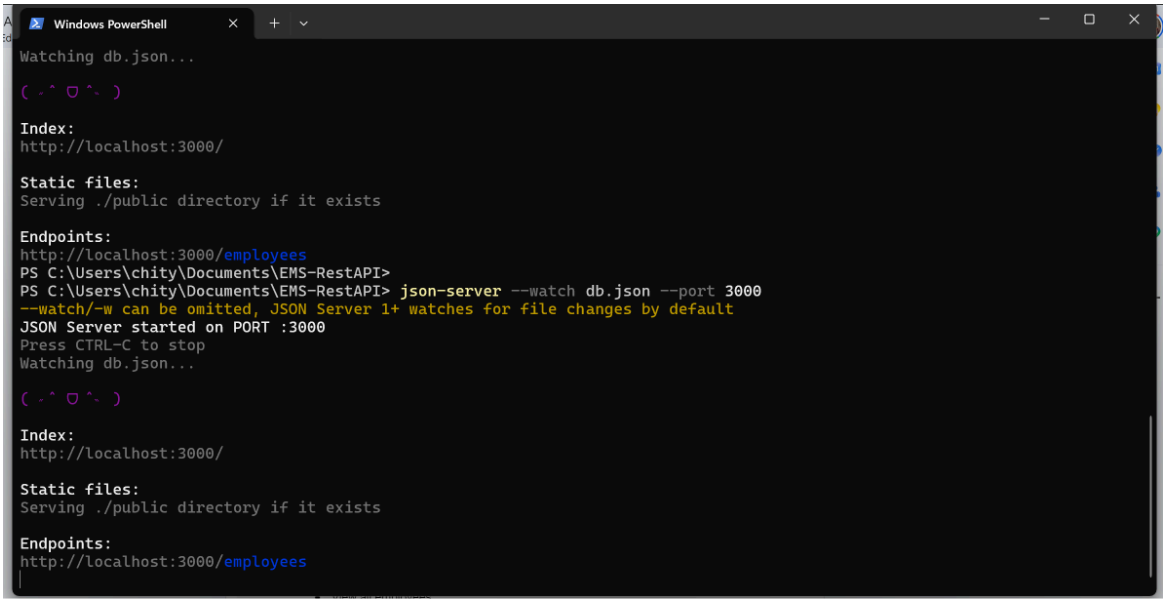

Figure 1: JSON Server running on port 3000 (Windows PowerShell)

5.2 Verifying the API in Browser

Navigating to http://localhost:3000/employees in the browser confirms that the API is serving the employee records from db.json.

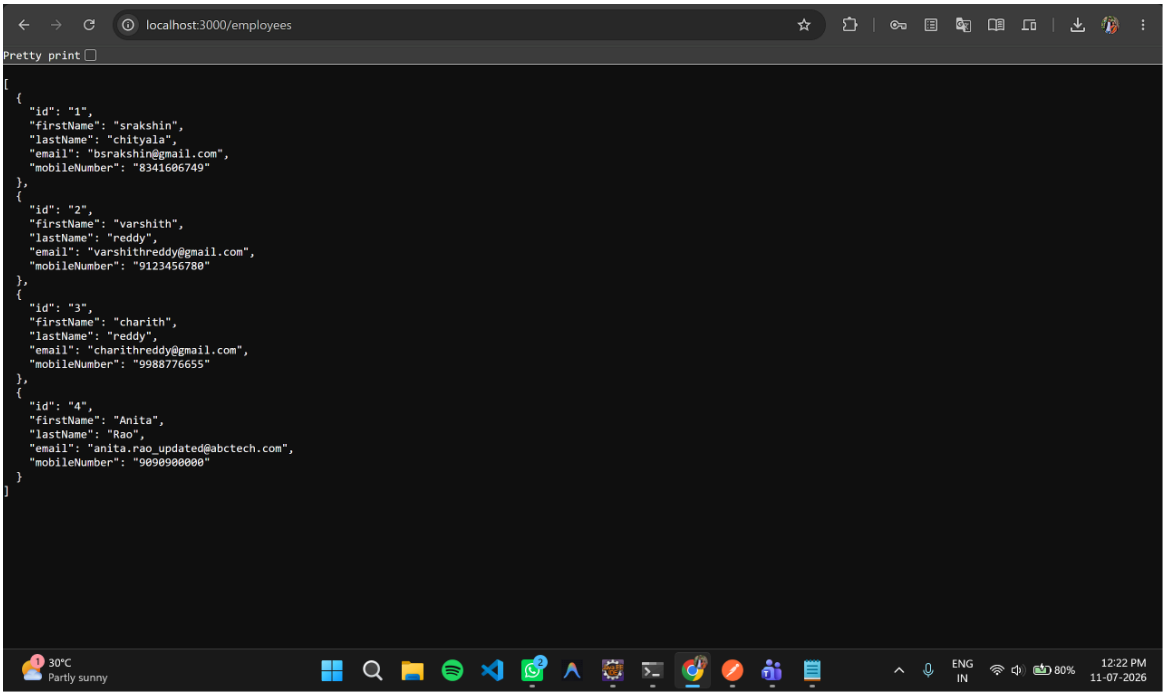

Figure 2: Employee data returned at localhost:3000/employees

6. REST API Endpoint Testing (Postman)

Each REST API endpoint was tested individually using Postman. The requests, responses, and status codes for each operation are documented below.

6.1 GET All Employees

Method: GET | URL: http://localhost:3000/employees

This request retrieves the complete list of employee records. The response returned a 200 OK status along with all employee records stored in db.json.

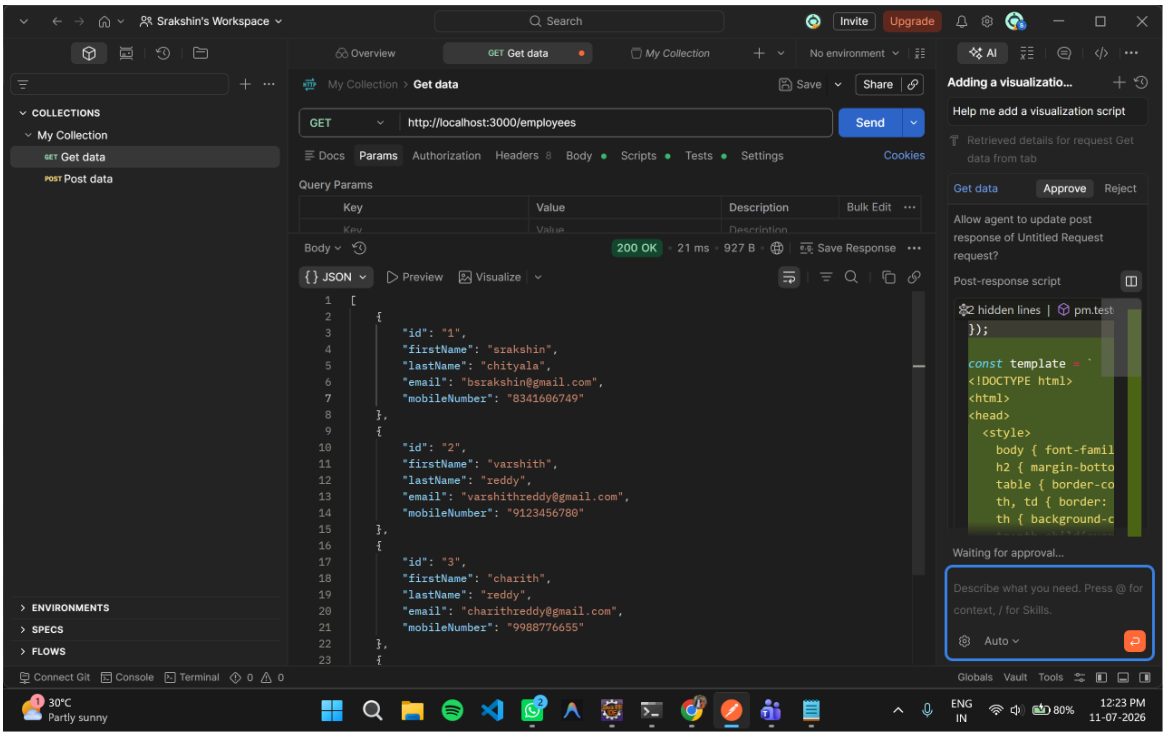

Figure 3: GET all employees — 200 OK

6.2 POST — Add New Employee

Method: POST | URL: http://localhost:3000/employees

A new employee record (Divya Kumar) was added by sending a JSON body in the POST request. JSON Server auto-generated a unique ID (a31P6gZV4dA) for the new record, and the response confirmed a 201 Created status.

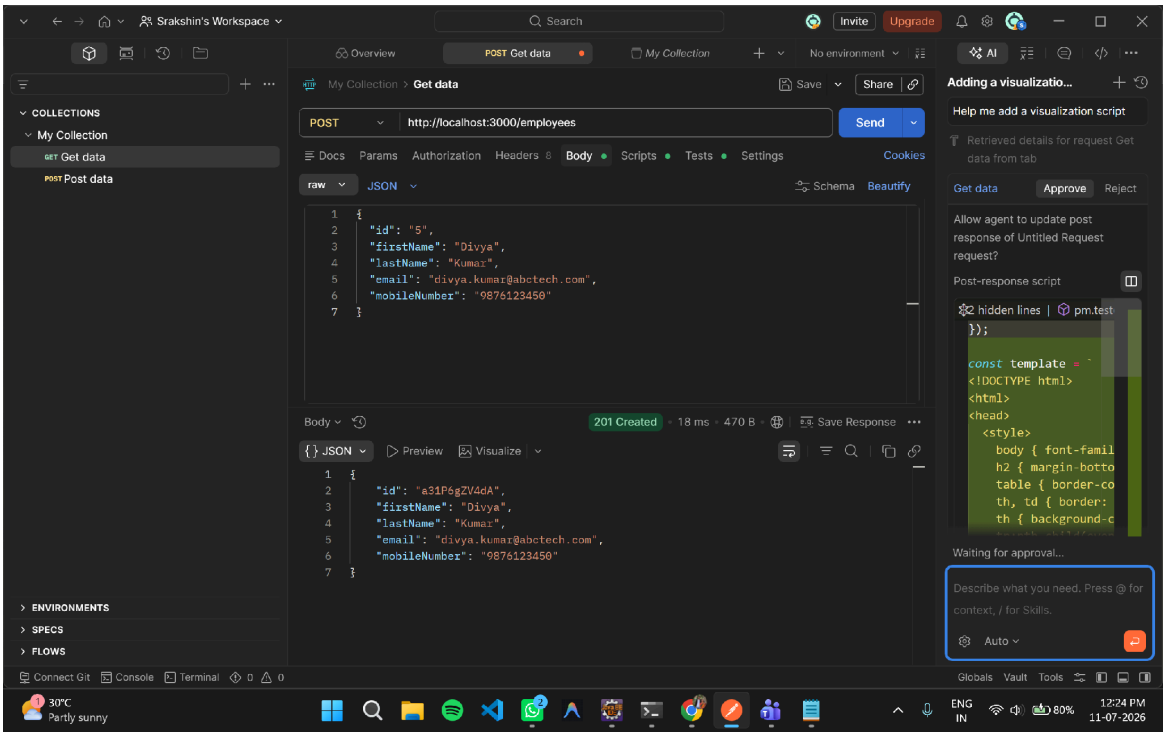

Figure 4: POST request to add a new employee — 201 Created

The GET all employees request was run again to verify that the new employee record was successfully added to the collection.

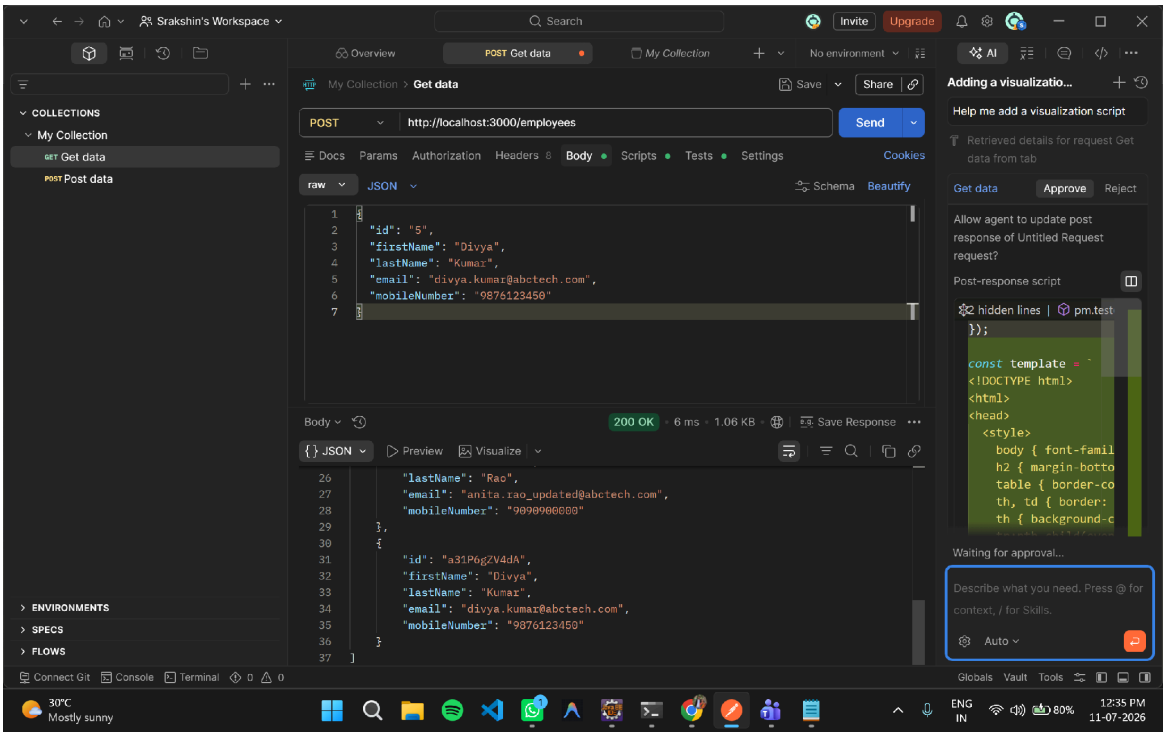

Figure 5: GET all employees showing the newly added record

6.3 PUT — Update Employee (Full Update)

Method: PUT | URL: http://localhost:3000/employees/2

The PUT request was used to fully update employee ID 2 (Varshith Reddy), replacing the email field. The server responded with 200 OK and the updated employee object.

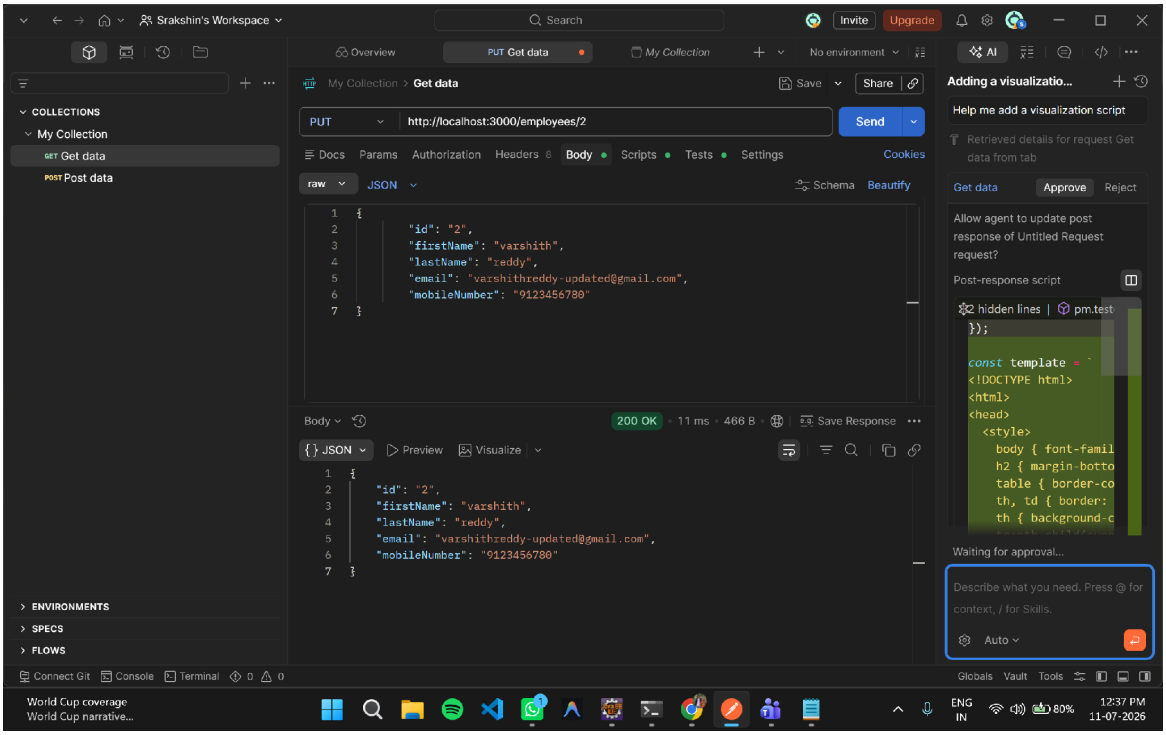

Figure 6: PUT request updating employee ID 2 — 200 OK

6.4 PATCH — Partial Update

Method: PATCH | URL: http://localhost:3000/employees/2

The PATCH request was used to update only the mobileNumber field of employee ID 2, without affecting the other fields. The server responded with 200 OK, confirming the partial update.

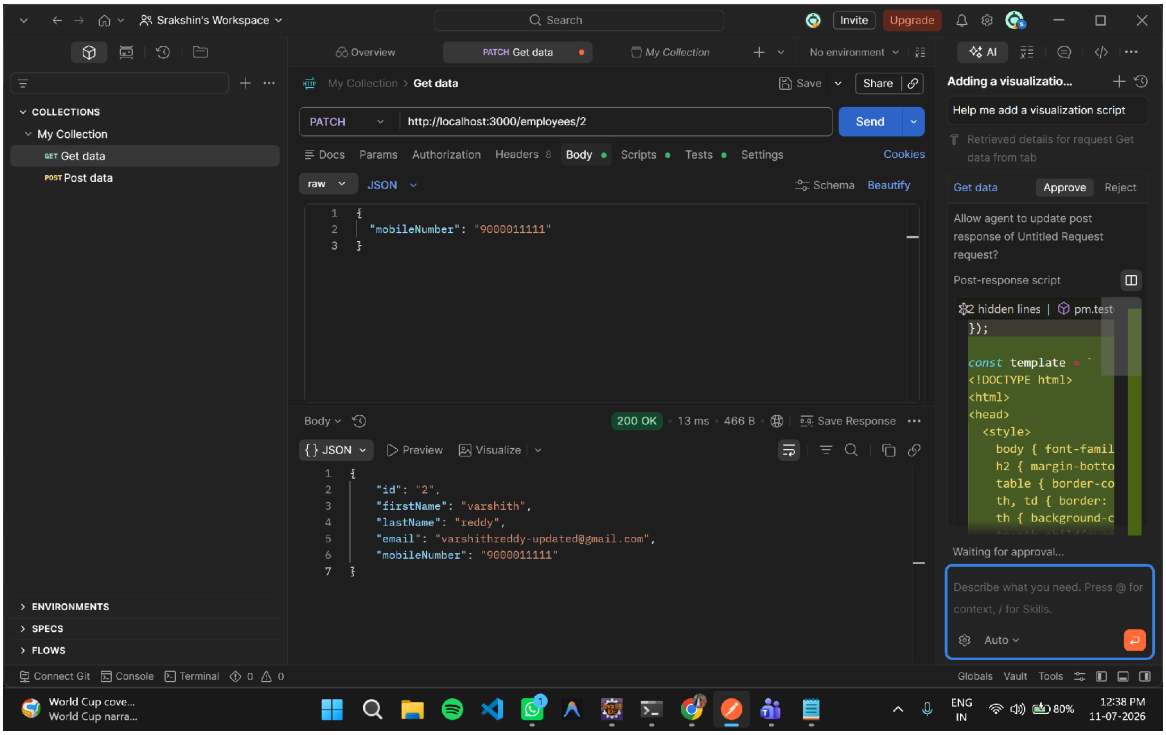

Figure 7: PATCH request updating mobile number — 200 OK

6.5 DELETE — Remove Employee

Method: DELETE | URL: http://localhost:3000/employees/2

The DELETE request removed employee ID 2 (Varshith Reddy) from the collection. The server responded with 200 OK, confirming the record was deleted.

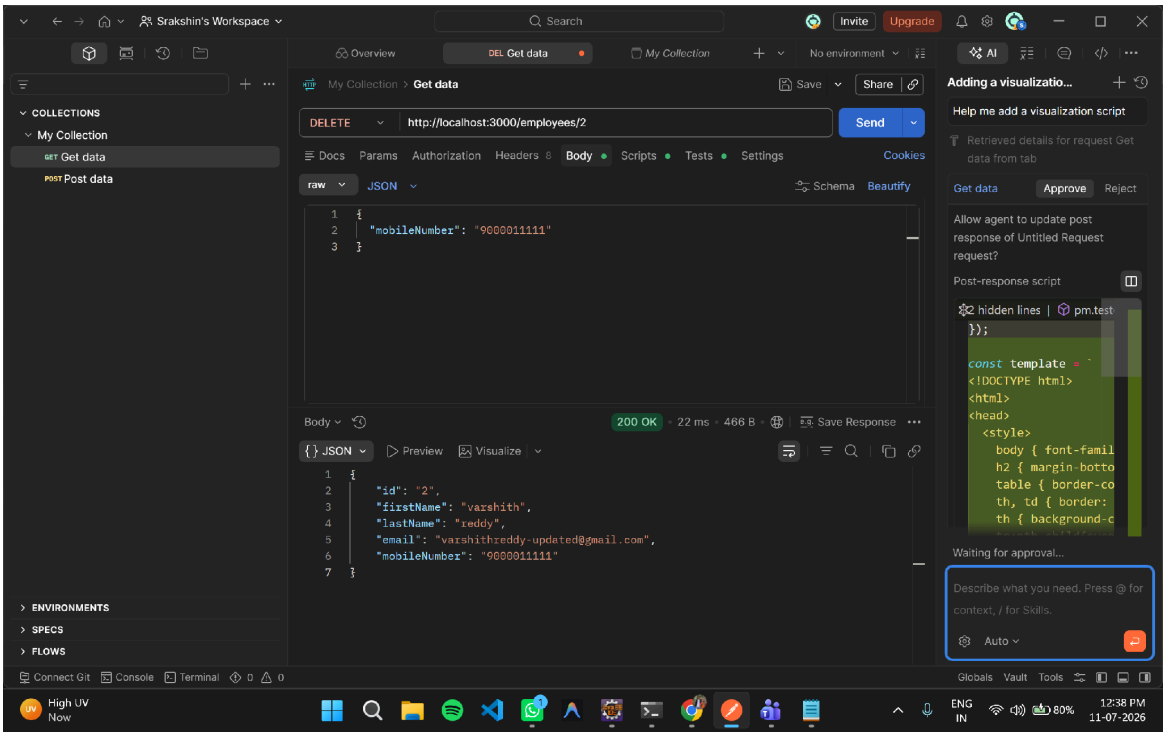

Figure 8: DELETE request removing employee ID 2 — 200 OK

6.6 Verifying Deletion

A final GET all employees request was executed to confirm that employee ID 2 no longer exists in the collection.

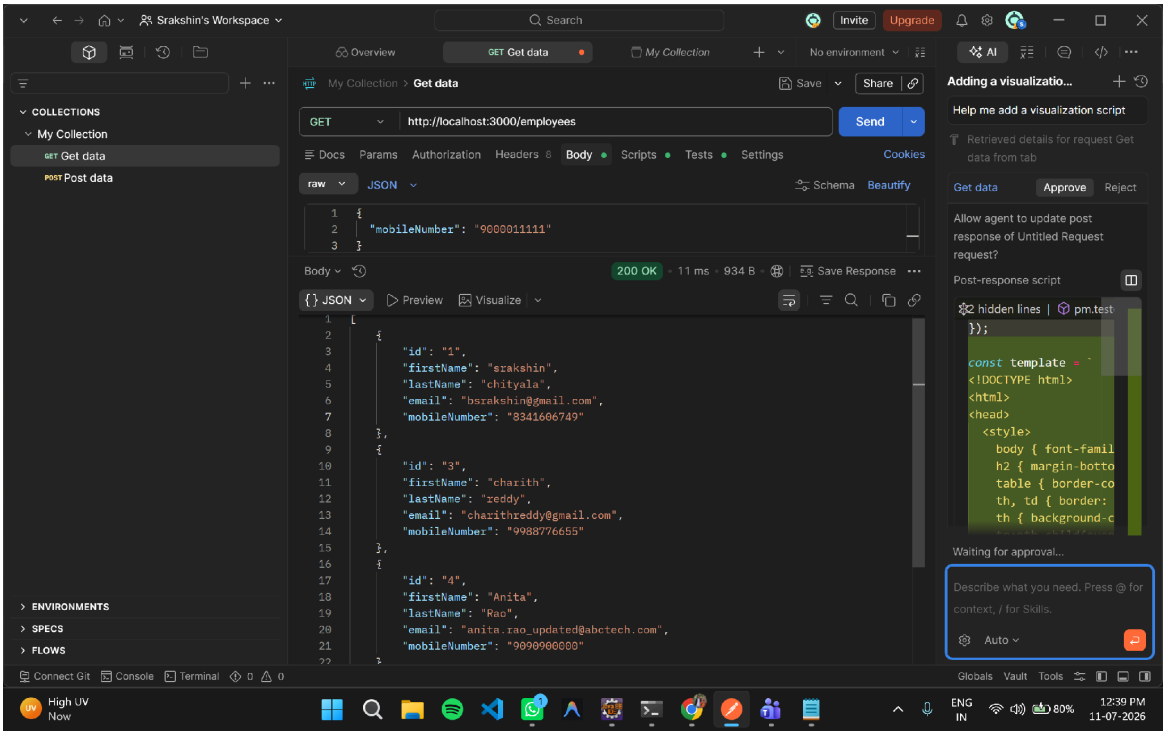

Figure 9: GET all employees confirming employee ID 2 was removed

7. Conclusion

All REST API endpoints (GET, GET by ID, POST, PUT, PATCH, and DELETE) for the Employee Management System were successfully configured using JSON Server and tested using Postman. The Fake REST API was hosted on port 3000 and supported all required CRUD operations on employee records, meeting the objectives outlined for the mini project.

8. Submission Checklist

- db.json file configured and hosted on port 3000
- All REST endpoints tested using Postman (GET, GET by ID, POST, PUT, PATCH, DELETE)
- Screenshots captured for each activity
- Document created with proper headings and screenshots
- Document submitted to repository
- Hands-On Tracker updated